

PRODUCT REQUIREMENTS DOCUMENT (PRD)

Contexto del Proyecto: ETL Solar Serverless - Procesamiento del Dataset PVOD

Destinatario: Agente de Programación (Antigravity AI) **Rol Asignado:** Tech Lead de Ingeniería de Datos

1. Resumen Ejecutivo

El propósito de este sistema nativo en la nube es la ingesta automatizada, limpieza matemática estricta, transformación algorítmica y disponibilidad analítica del Photovoltaic Power Output Dataset (PVOD). El valor de negocio principal es habilitar el monitoreo y proporcionar métricas para modelos precisos de pronóstico de potencia fotovoltaica (NWP y LMD), fundamentales para el balanceo de carga en tiempo real y la liquidación en mercados mayoristas de energía. El sistema culmina exponiendo métricas agregadas mediante una API de alto rendimiento.

2. Arquitectura del Sistema (GCP)

La arquitectura está diseñada para operar de forma 100% serverless en Google Cloud Platform (GCP) bajo principios de Clean Architecture:

- **Orquestación:** Cloud Scheduler o Cloud Tasks para la invocación asíncrona y orquestación de la carga del pipeline ETL.
- **Procesamiento y API (Cloud Run):** Ejecución de los contenedores Docker inmutables que alojan tanto las rutinas asíncronas del ETL como el microservicio de la API RESTful (FastAPI). Escala automáticamente desde cero instancias hasta miles en segundos.
- **Almacenamiento Intermedio (Cloud Storage):** Actúa como *buffer* (nivel oro/gold layer) alojando temporalmente los datos extraídos y serializados en formato binario Apache Parquet antes de su ingesta.
- **Data Warehouse (BigQuery):** Persistencia transaccional ACID de los datos procesados con esquemas estrictamente tipados y estructuración en almacenamiento columnar.

3. Diseño de Software y Patrones

La base de código debe particionarse según la Regla de Dependencia en capas aisladas: Entidades (Dominio), Casos de Uso (Aplicación), Adaptadores de Interfaz y Frameworks/Drivers. El agente de IA deberá utilizar **Polars** en lugar de Pandas (debido al

escalado eficiente y *lazy evaluation* en la nube) y emplear el módulo integrado `abc` de Python para aplicar obligatoriamente los siguientes patrones:

- **Patrón Factory (Ingesta):**
 - **Contrato:** Crear clase `PVODExtractionPipeline(ABC)` con el método `@abstractmethod extract_data_to_buffer()`.
 - **Implementaciones:** `GitHubRawExtractor` (prioritario vía raw requests a `raw.githubusercontent.com`) y `ScienceDBHttpExtractor` (contingencia/fallback vía API HTTP directa).
 - **Constructor:** `ExtractionFactory`, que evalúe variables de entorno y devuelva dinámicamente el objeto de extracción sin anidar declaraciones.
- **Patrón Adapter (Data Warehouse):**
 - **Contrato:** Puerto abstracto `DataWarehouseRepository(ABC)` en la capa de Aplicación exigiendo `load_dataframe_idempotent()`.
 - **Implementación:** `BigQueryAdapter` en la capa de Infraestructura que envuelva el cliente nativo de Google Cloud.
- **Patrón Strategy (Limpieza de Datos Fotovoltaicos):**
 - **Contrato:** Interfaz `SolarDataCleaningStrategy(ABC)` con el método `apply_cleaning(dataframe)`.
 - **Implementaciones (Estrategias concretas):**
 1. `MissingValueImputerStrategy`: Imputación basada en splines o curva base de irradiancia teórica extraterrestre para resolver lagunas temporales.
 2. `HampelFilterStrategy`: Filtro por desviación absoluta mediana de ventana móvil para anomalías en la velocidad del viento.
 3. `NighttimeZeroingStrategy`: Cálculo de elevación de ángulo solar para forzar irradiancia y salida de potencia exactamente a cero entre la puesta y salida del sol.

4. Esquema de Datos y Reglas de Negocio

- **Estructura del Dataset PVOD:** 271,968 registros con un intervalo estricto de 15 minutos (Tiempo Estándar Local, LST), pertenecientes a 10 estaciones fotovoltaicas a lo largo de 348 días (1 de julio de 2018 al 13 de junio de 2019).
- **Alineamiento (NWP y LMD):** Correlacionar predicciones macroescalares (Irradiancia global, T° bulbo seco, dirección/velocidad viento, etc.) con mediciones locales microclimáticas (Irradiancia difusa, presión atmosférica, y salida de potencia en MW). El ETL orquesta *Joins* asíncronos para alinear las marcas de tiempo sin desviación.
- **Restricciones Matemáticas:** La irradiancia global no puede ser negativa ni exceder la constante solar extraterrestre. Abortar ejecución inmediatamente si fallan las validaciones físicas, en lugar de aplicar políticas de reintento.
- **Garantías ACID e Idempotencia:** Sustituir la autogeneración de UUIDs aleatorios por un identificador determinista (MD5) creado usando el módulo `hashlib`: `job_id = "pvod_load_" + hashlib.md5(source_uri + project_id +`

`dataset_id + target_table + hash_contenido_parquet).hexdigest()`. Inyectar esto en `jobReference.jobId` del adaptador para que la API evite la duplicación de inserciones en caso de reintentos asíncronos.

- **Particionamiento y Clustering en BigQuery:** Configurar `time_partitioning_field` a nivel diario o mensual anclado en la columna de timestamp local LST, aplicando un `clustering` secundario apoyado en el campo `station_id`. Exportar de Polars obligatoriamente a formato Apache Parquet (con compresión RLE y diccionarios) antes de realizar el Load Job masivo a BigQuery.

5. Requerimientos No Funcionales

- **Configuración del Dockerfile:** Implementar *multi-stage builds*. Utilizar de manera exclusiva e innegociable la imagen base `python:3.11-slim-bullseye` (o equivalente Debian slim que usa `glibc`), ya que Alpine Linux (`musl libc`) rompe la compatibilidad binaria con librerías compiladas en C para datos como Polars o PyArrow. El contenedor purgado correrá vía `ENTRYPOINT` con `uvicorn` (ej. puerto 8080).
- **Entorno y Manejo de Secretos:** Gestionar dependencias con el paquete basado en Rust `uv` emitiendo un archivo `requirements.txt` matemáticamente determinista con pines criptográficos. Archivo estricto `.gitignore` (excluyendo obligatoriamente `.env`, `__pycache__`, `.venv`, y artefactos de datos temporales `*.csv`, `*.parquet`, `*.json`). En Cloud Run de producción, inyectar variables en memoria vía Google Cloud Secret Manager, jamás dentro de la imagen estática.
- **Observabilidad y Formato de Logging:** Abandonar logs en texto plano. Implementar un `CloudLoggingHandler` (*JSON Structured Logging*) emitido a `stdout` integrado con Google Cloud Logging. Requerimientos clave para la estructura JSON: llaves reservadas `severity`, `message`, `logging.googleapis.com/trace`, y un diccionario `attributes` adjuntando métricas vitales como `records_processed`, `anomalies_detected`, y `station_id`. Jerarquía de manejo de errores obligatoria (e.g., `SolarETLError`, heredado a `BigQueryConnectionError`, `IrradianceOutOfBoundsError`) eludiendo las cláusulas amplias de captura (`except Exception as e`).

6. Milestones de Desarrollo

- **Fase 1: Aprovisionamiento de Infraestructura, Seguridad y Componentes de Ingesta**
 - **Tarea 1.1:** Inicializar entorno con gestor `uv`, crear `.gitignore` y configurar estructura de directorios bajo Clean Architecture.
 - **Tarea 1.2:** Implementar capas de Ingesta usando el Patrón Factory (cliente HTTP a espejos GitHub como fuente primaria y scidb.cn de contingencia) y descargar CSV hacia un buffer de memoria o GCS temporal.

- **Fase 2: Transformación Analítica, Fusión Numérica y Perfilado**
 - **Tarea 2.1:** Carga perezosa (*lazy query* vía `scan_csv`) en entorno Polars. Ejecución de *Joins* temporales alineados cada 15 minutos exactos para cruzar matrices LMD y NWP.
 - **Tarea 2.2:** Aplicación estricta del Patrón Strategy para la purga heurística (Hampel Filter, Zeroing nocturno, Missing Value Imputing).
 - **Tarea 2.3:** Aplicación estricta de Data Types de Polars y volcado final binario altamente comprimido en Apache Parquet a GCS (Capa Oro).
- **Fase 3: Integración Transaccional Resiliente y Despliegue de Observabilidad**
 - **Tarea 3.1:** Activación global de manipuladores estructurales nativos de JSON de Google Cloud Logging.
 - **Tarea 3.2:** Construcción del módulo de cifrado de hash determinista (MD5) del *job_id* y ejecución atómica del BigQuery Load Job utilizando clústeres y particiones geográficas/temporales mediante cliente asíncrono.
- **Fase 4: Contenedorización Final, Despliegue API Serverless y Servicio de Consulta**
 - **Tarea 4.1:** Diseño del microservicio de API con FastAPI. Emplear modelos tipados en Pydantic y consultas SQL puramente parametrizadas (`ScalarQueryParameter`) a BigQuery (prevención de barridos completos o inyección SQL), inyectando el cliente de red como dependencia de arranque de FastAPI para reciclar conexiones.
 - **Tarea 4.2:** Construir y purgar imagen Docker *multi-stage* sobre Debian base y subir a Artifact Registry.
 - **Tarea 4.3:** Despliegue final a Google Cloud Run (asegurando el auto-escalado horizontal desde y hacia cero), validación final extremo a extremo de las métricas agregadas del PVOD en la nube.